Source-Grounded CRM Extraction: Progress Update

Chuyue Wang, November 2026
EAS 0099, Fall 2026. Faculty supervisor: Professor Benjamin C. Pierce.

Question

A document may say that someone has a new role, or that one company now owns another. Before a customer relationship management (CRM) system can use a fact like this, something has to extract it, find the right record, check the evidence and dates, and decide whether to accept the update, send it for review, or reject it. I want to find out whether splitting these steps apart leads to fewer incorrect writes, and how many useful updates that costs.

This question connects to earlier work. Sending a doubtful case to review is a reject option in the sense of Chow [1], and balancing accuracy on accepted cases against coverage of useful ones is similar to the tradeoff in selective classification [2]. This update covers the design, the dataset, and my results on the 66 development cases.

Workflows

I compare the two workflows in Figure 1, which I call direct and staged below. They use the same small local model and get the same sources, CRM, extraction instructions, and update policy, and both send accepted updates to the same writer. Direct asks the model to do everything and follow the policy in one response. Staged applies the policy in Python, so the model only extracts a candidate and then checks the evidence, date, and source agreement.

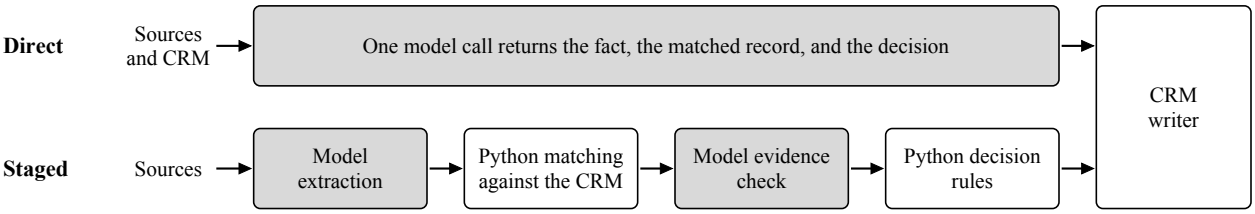


Figure 1. The two workflows. Gray boxes are model calls and white boxes are Python code. Both share one CRM writer.

I based the staged design on earlier work. Wu et al. [3] split a task into chained steps with visible results, and Khot et al. [4] show that a step can be ordinary code instead of a model. The evidence check labels a claim as supported, uncertain, or contradicted, which is close to the labels in the FEVER dataset [5], and it runs as a separate call, like the verification step in Chain-of-Verification [6]. Requiring an exact quotation from an allowed source is a simple, rule-based version of the attribution that Rashkin et al. [7] and Gao et al. [8] measure in generated text. Matching against the CRM is a small entity-resolution problem [9]. Its three outcomes (matched, ambiguous, and new) are like the link, possible link, and non-link of Fellegi and Sunter [10], and ambiguous cases go to a reviewer.

Data and setup

Dataset. I use 132 cases, half for development and half for test. Each half has 38 cases that should be accepted, 17 that should go to review, and 11 that should be rejected. Sixty cases adapt passages from FewRel [11] (Table 1), a public dataset whose Wikipedia sentences were aligned with Wikidata [12] facts by distant supervision [13] and then filtered by crowdworkers. These passages cover position held, ownership, and headquarters, and their CRM records, duplicates, and permission labels were built for this project. I chose FewRel because its short passages fit a task with one fact per case. DocRED's multi-sentence documents [14] and CrossRE's generic relation types [15] would need more adaptation. The other 72 cases are synthetic documents written for this project. They add things the public passages do not have, such as facts that have gone stale [16], sources that disagree [17], tentative claims, and denials. No target identity or passage appears in both halves, and the synthetic document forms differ.

Table 1. Open resources used in this project.

Resource	Used for	License	Source
FewRel 1.0 [11]	60 public passages	MIT	github.com/thunlp/FewRel
Qwen3-4B, 4-bit [18]	The model in both workflows	Apache 2.0	huggingface.co/mlx-community/Qwen3-4B-4bit
MLX [19]	Running the model on Apple silicon	MIT	github.com/ml-explore/mlx

Model and runs. Both workflows use Qwen3-4B [18] in 4-bit form on MLX [19], with greedy decoding, thinking turned off, and a 512-token limit. I ran each workflow once on each development case in a shuffled order, which gives 132 executions. I also ran six cases a second time under both workflows, which adds 12 more. Each execution starts from a fresh copy of the CRM, and the model never sees the answer keys.

Measures. *Correct writes / accept decisions* is the share of accept decisions that save the correct value in the CRM. *Useful updates written* counts correct writes among the 38 cases with a valid update. *Review/reject cases accepted* counts accept decisions among the 28 cases that should go to review or be rejected. *Correct decisions* counts right accept, review, or reject decisions, and an invalid response counts as wrong. *Invalid responses* fail validation and write nothing. I give 95% Wilson intervals [20] for rates and use an exact McNemar test [21] for paired comparisons.

Development results

Table 2 shows the main results. Staged made more correct decisions and accepted none of the review/reject cases, but it wrote 3 fewer useful updates. When I compare the workflows case by case, staged alone made the correct decision on 25 cases and direct alone on 6 ($p < 0.001$). Direct alone accepted 21 review/reject cases and staged alone accepted none ($p < 0.001$). Direct alone wrote 8 useful updates and staged alone wrote 5 ($p = 0.58$), so this last difference could be chance. With only 38 useful cases, the 95% interval for direct's advantage runs from 4 fewer to 10 more updates, so I cannot say from these cases how large the cost is.

Table 2. Main results on the 66 development cases, with 95% Wilson intervals. Staged accepts fewer cases, and more of the updates it accepts are correct.

Measure	Direct	Staged
Correct writes / accept decisions	29/57, 50.9% (38%–63%)	26/31, 83.9% (67%–93%)
Useful updates written	29/38, 76.3% (61%–87%)	26/38, 68.4% (53%–81%)
Review/reject cases accepted	21/28, 75.0% (57%–87%)	0/28, 0.0% (0%–12%)
Correct decisions	36/66, 54.5% (43%–66%)	55/66, 83.3% (73%–90%)
Invalid responses	9	4

Two things matter here. The development cases guided the design, so the numbers do not predict how the workflows do on new documents. That is what the test cases are for. The answer key also gained three valid value forms after a review of the development outputs, raising correct writes from 27 to 29 for direct and 23 to 26 for staged.

One result stands out. Every valid direct response was an acceptance. The prompt gives the same decision policy that staged applies in code, but direct never chose review, and its only 4 rejections came in invalid responses. It accepted every ambiguous identity, source conflict, duplicate record, future appointment, same-date conflict, and stale update, and 4 of 5 disallowed sources. For example, it overwrites Stefan Mensah's newer CRM role with a 2025 appointment, while staged rejects the update. Eight of direct's 9 invalid responses fail only on the match, which staged never asks the model for. Staged wrote 16 of 24 useful updates on FewRel passages and 10 of 14 on synthetic documents, and direct wrote 20 and 9.

I also scored the steps inside each workflow. Entity resolution was correct in 57 of 66 cases for staged and 48 of 66 for direct. All 56 staged quotations were found in their cited sources, compared with 63 of 64 for direct. All 31 staged accepts came from allowed sources, compared with 53 of 57 for direct. The verifier's flags were right on 40 of 42 correctly extracted candidates. In Table 3, the biggest difference is in organizations. Direct returned 12 that do not match the answer key, and staged missed 8 of the 30 expected.

Table 3. Extracted fields on the development cases. Precision is correct over returned, and recall is correct over expected.

Field	Direct		Staged	
	Precision	Recall	Precision	Recall
Names	62/64	62/64	55/56	55/64
Roles	37/41	37/40	36/36	36/40
Relationships	19/23	19/24	19/20	19/24
Organizations	30/42	30/30	22/24	22/30
Effective dates	26/26	26/26	26/29	26/26

What each stage contributes

An overall accuracy number can hide where a gain comes from [22], so I rescored parts of the workflow separately (Table 4). “Python steps” means matching and the decision rules. The first four rows use only the saved responses. The last row uses the answer key's candidate, so I can see how the later steps do without extraction errors.

Table 4. Results as parts of the workflow change. Rows 1 to 4 go from direct to full staged. Row 5 uses the answer key.

Workflow	Correct writes / accept decisions	Useful updates written	Review/reject cases accepted	Correct decisions
Direct	29/57	29/38	21/28	36/66
Direct's candidate, Python steps, no verifier	25/37	25/38	7/28	51/66
Staged extraction, Python steps, no verifier	26/33	26/38	2/28	53/66
Full staged workflow	26/31	26/38	0/28	55/66
Answer-key candidate, model verifier, Python steps	38/39	38/38	1/28	65/66

When I moved the match and the decision into Python, using direct's own candidates and no verifier, accepted review/reject cases dropped from 21 to 7. Dropping the model's match also makes 8 of direct's 9 invalid responses usable, and 3 of the 7 come from those. Useful updates fell from 29 to 25, with 6 lost and 2 gained. All 6 losses are historical roles where direct filled in an organization that the CRM record lacks, and matching sends those to review.

Using a separate extraction prompt then brought accepted cases down to 2, although 4 of those 5 stops came from the missing organization problem I describe next. Adding the verifier brought them to 0 without losing any useful updates. So the Python matching and rules do most of the work, which makes sense because direct does not apply the policy it is given.

The zero does not mean that every check worked as I intended. Only 19 of the 28 review/reject cases stopped at the rule written for them. Another 7, including both denials, stopped because the extractor left out the organization, which sends a current role to review before the verifier's answer is used.

To test the later steps without extraction errors, I ran the verifier on the answer key's candidates, which took 64 new model calls. The workflow then made 65 of 66 correct decisions, and its one error accepts a draft staff record as settled. The full workflow made 55, so most of the staged errors come from extraction. The verifier does not catch these errors, since it reported no problem with any of the 5 wrong candidates that staged wrote. I should also note that the decision measure favors any workflow that follows the policy exactly.

Where the staged workflow fails

On the useful cases, staged made 5 incorrect writes and missed 7 other updates (Table 5). The wrong writes come from extraction details that the verifier accepts as supported. The missed updates come from extraction responses that were invalid, empty, or incomplete.

Table 5. All 12 staged failures on the 38 useful cases, grouped by cause.

Staged failure	Wrong writes	Missed updates	Example
Effective date not stated in the source	3	0	“August 2015” stored as 2015-08-01
Duplicate identity	1	0	“UAL Corporation” added beside the UAL record
Wrong attribute	1	0	“working” stored as a role for an employer update
Entity type does not fit the attribute	0	3	“organization” returned as the entity type of a company
No candidate returned	0	3	No role found in “Attorney General Jeff Sessions”
Organization left out	0	1	“BL” not returned as Pavel Reeve's organization

Consistency

Every repeated execution returned the same responses as its first run (12 of 12). I also reran the development cases on a second Mac, an M5 Max, and the results do differ between the two machines, whose package versions also differ slightly. On the M5 Max, 98 of 144 executions returned identical responses, and 136 gave the same candidate, match, and decision. Correct writes / accept decisions were 28/59 for direct and 25/30 for staged, and no main count moved by more than 2 cases. On the M5 Max, the first four rows of Table 4 gave 23, 7, 2, and 0 accepted review/reject cases. This tells me the results are stable across machines, but it says nothing about new cases.

Cost

Staged used 4% more input tokens and 22% more output tokens.

Status and next steps

The dataset, both workflows, the evaluation code, the development analysis, and a demonstration on saved cases are ready. My repository at github.com/StevenWang-CY/source-grounded-extraction has the code, data, saved runs, and more detailed tables.

I will continue to implement and experiment, and I will present the 66-case test comparison, its error analysis, and the regression tests in my final report on December 10.

References

[1] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

[2] Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. In *NeurIPS*, 2017.

[3] T. Wu, M. Terry, and C. J. Cai. AI Chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *CHI*, pp. 1–22, 2022.

[4] T. Khot et al. Decomposed prompting: A modular approach for solving complex tasks. In *ICLR*, 2023.

[5] J. Thorne et al. FEVER: A large-scale dataset for fact extraction and verification. In *NAACL*, pp. 809–819, 2018.

[6] S. Dhuliawala et al. Chain-of-verification reduces hallucination in large language models. In *Findings of ACL*, pp. 3563–3578, 2024.

[7] H. Rashkin et al. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023.

[8] T. Gao, H. Yen, J. Yu, and D. Chen. Enabling large language models to generate text with citations. In *EMNLP*, pp. 6465–6488, 2023.

[9] V. Christophides et al. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys*, 53(6):1–42, 2020.

[10] I. P. Fellegi and A. B. Sunter. A theory for record linkage. *J. Amer. Statist. Assoc.*, 64(328):1183–1210, 1969.

[11] X. Han et al. FewRel: A large-scale supervised few-shot relation classification dataset with state-of-the-art evaluation. In *EMNLP*, pp. 4803–4809, 2018.

[12] D. Vrandečić and M. Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.

[13] M. Mintz et al. Distant supervision for relation extraction without labeled data. In *ACL-IJCNLP*, pp. 1003–1011, 2009.

[14] Y. Yao et al. DocRED: A large-scale document-level relation extraction dataset. In *ACL*, pp. 764–777, 2019.

[15] E. Bassignana and B. Plank. CrossRE: A cross-domain dataset for relation extraction. In *Findings of EMNLP*, pp. 3592–3604, 2022.

[16] B. Dhingra et al. Time-aware language models as temporal knowledge bases. *Transactions of the ACL*, 10:257–273, 2022.

[17] H.-T. Chen, M. Zhang, and E. Choi. Rich knowledge sources bring complex knowledge conflicts: Recalibrating models to reflect conflicting evidence. In *EMNLP*, pp. 2292–2307, 2022.

[18] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[19] A. Hannun et al. MLX: Efficient and flexible machine learning on Apple silicon. Software, 2023.

[20] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *J. Amer. Statist. Assoc.*, 22(158):209–212, 1927.

[21] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[22] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan. AI agents that matter. arXiv:2407.01502, 2024.